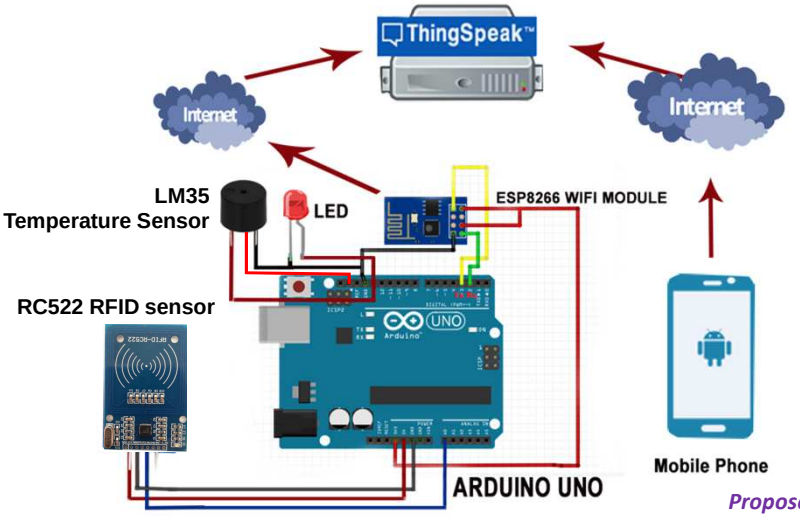
Faculté des sciences de Monastir
Niveau LSI3

Année Universitaire 2025-26

# Matériel et logiciel pour faire de l'IoT




*Proposé par Mr Samir Belaid*

## 1. Matériel & logiciel

- Un **objet IOT** est divisé en **deux parties**:
  - **Matériel**
  - **Logiciel**

### *1.1 Matériel*

Il est fait de composants électroniques (ou électriques) et composé de **3 parties**:

1. Capteurs / actionneurs
2. Microcontrôleur ou micro-ordinateur
3. Connexion réseau

#### ① Capteurs/Actionneurs

- **Capteur** : un dispositif qui transforme un phénomène physique en un signal électrique. *Exemple*: Chaleur, humidité, lumière, etc…
- A**ctionneur** : Un dispositif qui transforme le signal électrique en un phénomène physique (le contraire d'un **capteur**).

## ① Capteurs/Actionneurs

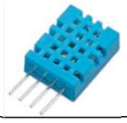
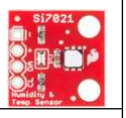
- Il existe des centaines voir des milliers de capteurs et d'actionneurs, allant d'un capteur de température à un  capteur de force et d'un moteur à une lampe.

**Comment choisir son capteur et/ou actionneur ?**

- Tout dépend des besoins du projet IOT, mais on peut se référer aux critères suivants :
  - Tension et courant de fonctionnement
  - Précision
  - Protocole de communication utilisé
  - Phénomène physique à capter ou générer (capteur/actionneur)
  - le prix

## Exemple1 : Capteur

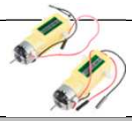
Notre objectif consiste à mesurer l'humidité de l'air avec une bonne précision. Comme tension de fonctionnement on a besoin de 5v. voici une liste des capteurs d'humidité trouvés sur internet.

| |  |  |  |  |
|---|---|---|---|---|
| **Nom** | DHT11 | DHT22 | HIH-4030 | Si7021 |
| **Prix** | 15DT | 28DT | 45DT | 24DT |
| **Tension/courant** | 3.3 à 6 V | 3.3 à 6 V | 4 à 5.8 V | 1.9 à 3.6 V |
| **Précision** | ± 5 % | ± 2 % | ± 3.5 % | ± 3 % |
| **Protocole de communication** | OneWire | OneWire | Analogique | i2C |

- Si on cherche la précision, il faut choisir le DHT22,
- Par contre, si on prefère le bas prix on a le DHT11.
- **P**our une **production de masse,** il est recommandé de choisir la **précision**.
- Aussi, dans le domaine de la **santé**, la **précision** prime devant le **prix**.

## Exemple2 : Actionneur

On souhaite faire un robot véhicule et on a besoin de 4 moteurs pour tourner les roues avec une tension de 5v. Dans ce cas, on ne cherche pas la **précision** mais plutôt la **vitesse** et la **légèreté**.

| |  |  |  |
|---|---|---|---|
| **Nom** | DG01D:48:1 motoréducteur | SM-428YG011-25 moteur pas à pas | Parallax continuous Rotation Servo |
| **Prix** | 15DT | 30DT | 60DT |
| **Tension/courant** | 4.5v/250mA | 12v/330mA | 4 à 6 v/140mA |
| **Vitesse de rotation** | 140 RPM | 9 RPM | 50 RPM |
| **Masse** | 30g | 20g | 42.5g |
| **Protocole de communication** | Digitalcontrol/normal | Digitalcontrol/normal | Digitalcontrol/PWM |

- Le motoréducteur DG01D:48:1 est le mieux adapter pour faire tourner le robot, car il a une vitesse de rotation élevée avec un bas prix et fonctionne à 5V avec une faible masse.
- Il faut noter que le choix est toujours guidé par l'idée du projet (le cahier des charges).

## ② Microcontrôleur/Micro-ordinateur

- Chargés de **recueillir** les **valeurs** provenant des **capteurs**, de faire un **traitement** sur les données et voir de **sauvegarder** une partie.
- Peuvent activer des **actionneurs** du genre: Faire tourner un moteur à 180°, allumer une lampe, etc…
- Il y a 2 types.
  - **Microcontrôleur**
  - **Micro-ordinateur**

### Microcontrôleur

- Equipé de **plusieurs ports** de connexion, **microprocesseur**, **RAM** et **ROM**, à la seule différence par rapport à un ordinateur, ses ressources sont très limitées, accès pour faire que des petites opérations.
- **Utile** pour tout ce qui est **acquisition** et **traitement** des données provenant des capteurs et **contrôle** des actionneurs.
- Il lui faut un programme informatique pour fonctionner.

## Comment choisir le microcontrôleur ?

- Pour choisir un microcontrôleur, on peut se référer aux éléments suivants:
  - **Nombre de port** d'acquisition de données et de contrôle
  - **Puissance de calcul** et la taille de la **mémoire**
  - **Tension** de fonctionnement et la **consommation** en énergie
  - **Protocoles** de communication
  - **Langage** de programmation supporté
  - le **prix**






| Nom | Arduino Uno | Node MCU (esp8266) | Pycom | Nucleo-F401RE (STM32) |
|---|---|---|---|---|
| Nombre de port | 20 | 16 | 22 | 50 |
| Tension / courant | 5v/20mA | 3.3 à 5 V/550mA mode wifi | 3.3 à 5.5V/ 400mA en 3,3v | 7V to 15V / 2.4uA at standby |
| Puissance et mémoire | Atmega328P / 32 KB | Tensilica Xtensa LX106 / 4MB | Espressif ESP32 SoC / 8MB | ARM Cortex M4 / 512KB |
| Interface réseau | NON | Wifi | Wifi, Bluetooth, Lora, Sigfox, LTE NB1 | NON |
| protocole compatible | I2C , SPI , UART, Analog, Digital | I2C ,I2S,SDIO, SPI , UART, Analog, Digital, ADC | I2C , SPI , UART, Analog, Digital, ADC | I2C , SPI , UART, USART , Analog, Digital, ADC |
| langage de programmation | Arduino, C/C++ | Arduino, C/C++, micropython, lua, javascript | Micropython | Arduino, C/C++, micropython |
| prix | 60DT | 30DT | 100DT | 40DT |

## Micro-ordinateur

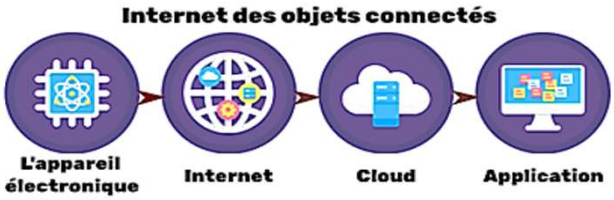
- A une **puissance** de calcul et une **mémoire** d'accès **proches aux ordinateurs** (Certains l'utilisent comme un ordinateur de bureau).
- Doté d'un **système d'exploitation** (linux la majeure partie).
- Souvent **utilisé** pour concevoir une **passerelle** (gateway).
- Equipé des **connectiques** pour accéder au **réseau** internet,
- Utilisé pour **recevoir** des données à partir des **capteurs** et les **envoyer** au **serveur cloud**.
- Le **choix** du micro-ordinateur peut être guidé par les caractéristiques suivantes:
  - **Puissance** de calcul et taille de la **mémoire**,
  - **Système d'exploitation**,
  - **Nombre de port** d'acquisition de **données** et de **contrôle**,
  - **Prix**.

## Micro-ordinateur

|  |  |  |  |
|---|---|---|---|
| **Raspberry pi 4** | **BeagleBone black** | **Omega2 Pro** | **Iattepanda Delta 432** |
| 40 | 92 | 30 | 50 |
| Quad core Cortex-A72 (ARM v8) / 4GO | AM335x 1GHz ARM Cortex-A8 /512 MB | 580MHz MIPS CPU / 128 MB | Intel 8th Gen Celeron Processor / 4 GB |
| Wifi, Ethernet, Bluettoth | Ethernet | Wifi, Ethernet | Wifi, Ethernet, Bluettoth |
| I2C , SPI , UART, Digital | I2C , SPI , UARTs, Digital, ADC | I2C, I2S, SPI , UART, Analog, Digital | I2C, I2S, USB, TTL, UART, RTC Digital, ADC |
| Ubuntu, raspbian, OSMC, Kalilinux | FreeBSD, Debian, android, ubuntu | OpenWrt 18.06 Linux | Windows, Linux |
| 180 DT | 180 DT | 150 DT | 480 DT |

**NB**: les micro-ordinateurs **peuvent être utilisés pour acquérir des données** comme les microcontrôleurs (mais le contraire est rarement).

## *1.2 Logiciel (software)*




- **Rappel:** Les 4 parties distingués de l'écosystème IOT sont:

### ① Appareil connecté

- Comme **langage de programmation**, il n'y a pas de limite. Le seul problème est que les librairies peuvent parfois ne pas exister pour certains langages. en général on utilise :
  - **Arduino (C/C++)** / micropython / java / javascript
- Comme **logiciel a installé** sur ordinateur:
  - **Arduino IDE** / Eclipse / Codeblock / PlatfomIO (Atom ou Vscode ou Vscodium )
- Comme **plateforme de simulation** on propose :
  - **Wokwi** (https://wokwi.com/)

## ② Internet (passerelle)

- La majorité des **micro-ordinateurs** disposent du **système d'exploitation Linux**. Exemple de distribution:
  - **Raspbian**
  - Ubuntu server
  - RaspBSD
  - Arch Linux ARM
  - Ubuntu Core
  - windows IOT Core
  - Domoticz

## ③ Cloud

- On fait appel souvent à des services en ligne telque : ***Amazon Web Services AWS***, ***Microsoft Azure***, ***IBM cloud***, ***Thinkspeaks***, etc.
- Mais n'empêche nous pouvons transformer la carte ***raspberry pi*** en ***serveur*** et installer des services comme:
  - **Node-red**
  - Mozilla webthings
  - blynk
  - iobroker
- Il y a aussi la possibilité de développer un ***serveur*** *from scratch* avec des langages comme:
  - **HTML**
  - Javascript
  - Mysql
  - PHP
  - Css
  - python/ jango
  - Mongodb
  - Apache

## ④ Application

- Il s'agit plus de la visualisation de données et le contrôle.
- On peut utiliser des services ou application déjà prête pour visualiser, comme *Thingspeak*, *mqtt dash*, etc…
- Il est possible aussi d'en créer des applications, voici quelques langages:
  - Application web = Html, Css, Javascript, PHP, Apache, etc
  - Mobile = Java, flutter, swift, Appinventor (très bien pour les débutant)

- **NB**: Comme sur le matériel, le choix des technologies réside sur le projet et son cahier des charges , il faudra choisir des technologies **faciles à maintenir**.

## 2. Projet tutoré

Un client vient vous voir avec un problème. Il a une **table de micro-jardinage** ou il veut **semer des légumes** pour sa famille, mais avec les obligations de la vie, il n'a pas le temps t'**entretenir sa table**, que faire ?

### Cahier des charges

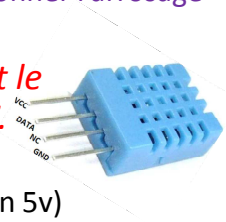
Il veut **automatiser sa table** pour qu'elle puisse s'**auto arroser** et lui **envoyer** des informations du climat relatives au milieu, sur son smartphone, ensuite enregistrer les informations dans une base de données, pouvoir prendre le contrôle de la pompe et que le tout fonctionne entre 3.3v et 5v et soit **autonome en énergie** avec un ***panneau solaire***. Sa **table de micro-jardinage** sera mise sur la terrasse et il souhaiterait utiliser le **WI-FI** pour recevoir les données.

## 2. Projet tutoré

***Cahier des charges*** : Le client veut des informations du climat.

- la température de l'air
- l'humidité de l'air
- l'humidité du sol
- le taux de luminosité
- pouvoir actionner l'arrosage

*Après consultation et **recherche sur le net**, voici le **matériel** et le **logiciel** à utiliser pour **désigner** sur mesure un **système IOT**.*




### Capteurs

- **DHT11** = Pour capter la température et humidité (protocole one wire, tension 5v)
- **Capacitive Soil Moisture** = Pour l'humidité du sol ( protocole Analogique, tension 3.3 à 5v)
- **Photorésistance** = Pour le taux de luminosité (protocole Analogique, tension 3.3v à 5v)

### Actionneur







- **Pompe** = Arrosage (tension 5v)
- **Relais** = Actionne la pompe (protocole Digital, tension 5v)



Pour les **capteurs** et l'**actionneur**, on a besoin de 4 ports au niveau du microcontrôleur et il doit supporté les protocoles (***onewire***, ***analogique***, ***digital***).

## Microcontrôleur et réseau de communication




- **ESP32**
  - Le ESP32 est le grand frère du NodeMcu (ESP8266) car ce dernier n'avait qu'un port analogique et d'après les capteurs on a besoin de 2 ports analogiques.
  - Il intègre le WIFI comme réseau de communication, pas besoin de **shield**, il fonctionnement entre 3.3v et 5v, donc c'est l'idéal.
- **Internet, la passerelle (gateway)**
  - **raspberry** pi zero w
  - Pour faire le **gateway** on peut choisir le **raspberry pi zero**, ces performances sont bonnes, il utilise **linux** comme **os** et consomme peu d'énergie avec un coût **pas cher**.

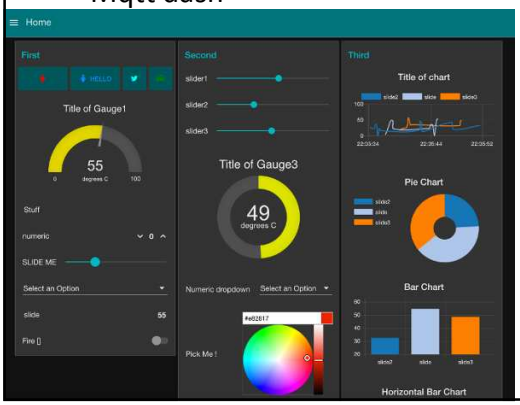


## Cloud (serveur)

- Node-red
- Mosquitto (mqtt protocole)
- Mongodb (base de données)

## Application

- Node-red dashboard
- Mqtt dash




- **NB**: R**aspberry** peut jouer 2 rôles, **Passerelle** et **serveur local**, **Node-red** est un **service très efficace** et consomme peu en ressources (on peut ajouter des plug-in pour faire de l'analyse et du traitement de données), **sauvegarder** dans une base de données et de la **visualisation**.

- Côté **applicatif:** on peut utiliser **node-red dashboard** pour avoir des **graphes** et des **jauges**, pour visualiser les donnés sur son navigateur web, l'application mobile **mqtt dash** peut être une alternative à **node-red dashboard** comme ça, avec son smartphone il pourra voir les données et **prendre des décisions**, comme *déclencher l'arrosage*.

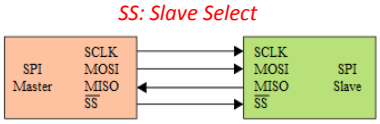
## 3. Initiation Arduino Uno

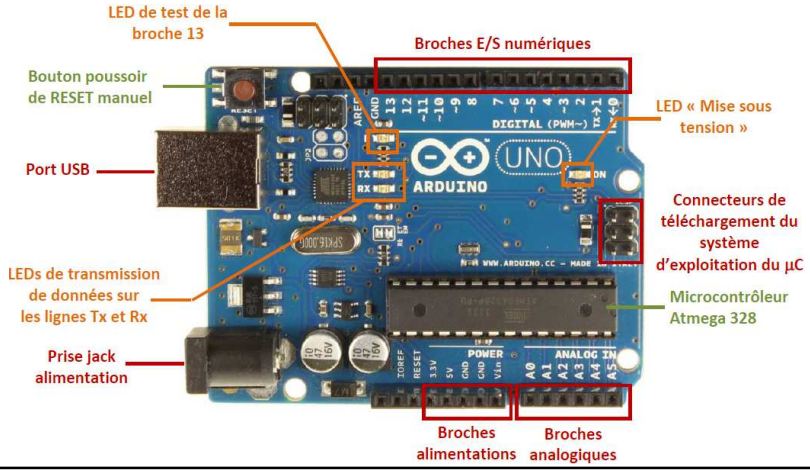
Arduino est une plate-forme de prototypage électronique open source basée sur du matériel et des logiciels flexibles et faciles à utiliser.

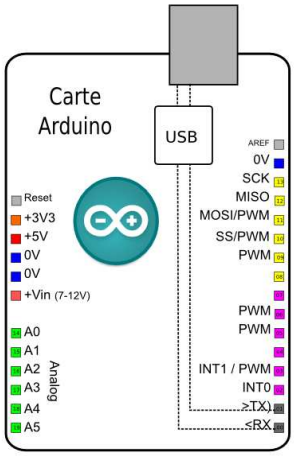
***Ports GPIO:*** *General Purpose Input/Output*
***Bus SPI:*** *Serial Peripheral Interface*
***PWM****: Modulation de largeur d'impulsion*

***Bus I²C:*** *Inter-Integrated Circuit*
***ICSP:*** *In Circuit Serial Programming*
***MISO****: Master In/Slave Out*

*SS: Slave Select*




## 3. Initiation Arduino

- Il existe plusieurs versions de cartes Arduino:
  - Cartes: *UNO, Mega2560, Mini, Nano, LilyPad, Fio, Pro, …*
  - Microcontrôleurs de *Atmel AVR*: *Mega328, Mega1280, Mega4809,…*
- Modules disponibles (*Shields*) avec les cartes Arduino



***Module Ethernet***



***Module Dragino LoRa Shield***



***Module Moteur***

- En plus de ces 3 modules, il existe d'autres modules, notamment :
  - Module GPS (Sparkfun)
  - Module SIM900 GSM GPRS Shield pour Arduino
  - Module Bluetooth HC05
  - etc...

## 3. Initiation Arduino

**Environnement de développement – Arduino IDE (https://www.arduino.cc/en/Main/Software)**

- Permet d'éditer, vérifier, compiler (verify/compile) et charger (upload) le *firmware* dans la mémoire du microcontrôleur,
- De dialoguer avec la carte lors de son fonctionnement (serial monitor)
- Il fournit des bibliothèques de code pour étendre les fonctionnalités.
- Sélectionner la liaison série COM_XX (menu Tools\serial_port)

**Programme Arduino:**

Le programme est au moins constitué de 2 fonctions:

- La fonction ***void setup()*** qui assure l'initialisation des E/S du microcontrôleur.
- La fonction ***void loop()*** **(**qui contient les instructions) est indéfiniment exécuté.

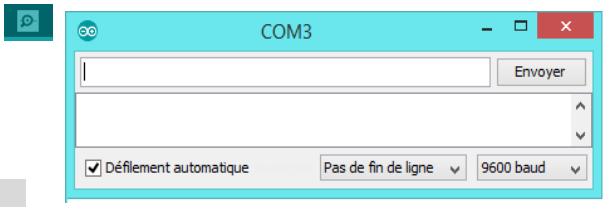
## 3.1 Téléverser votre premier programme !

- Le matériel nécessaire : PC+Carte Arduino+cable USB.
- Le programme ***Blink*** (*clignoter*) fait clignoter une LED sur la carte Arduino ! (reliée au pin digital 13).
- Cette LED permet de vérifier qu'une carte Arduino fonctionne bien.
- Voici le programme :

```
void setup()
{    //On initialise le pin 13 qui est associé à la LED en sortie
   pinMode(13, OUTPUT);
}     //Fin de la fonction setup()
void loop()
{          //La fonction loop() se répète à l'infini une fois la fonction setup() exécutée
 digitalWrite(13,HIGH);     //Place le pin digital 13 à l'état HAUT (5V) -> Led allumée
 delay(500);  //Met en pause le programme pendant la valeur de 500 en ms
 digitalWrite(13,LOW); //Place le pin digital 13 à l'état BAS (0V) -> Led éteinte
 delay(500);  //Met en pause le programme pendant la valeur de 500 en ms
}              // Fin de la fonction loop() => on retourne au début de la fonction loop()
```

## Faire communiquer le PC et la carte Arduino: Liaison série

- La liaison Série est un protocole de communication entre le PC et la carte Arduino via le câble USB permettant un échange d'informations via un **moniteur série** !
- Le **moniteur série** est accessible depuis le logiciel Arduino en cliquant sur la **loupe** en *haut à droite de la fenêtre* :




*9600 baud: Fréquence de transmission du moniteur série.*

- Pour activer et configurer la liaison série. Ajouter dans la fonction *setup()* la ligne:

```
Serial.begin(9600);
```

- Pour que la carte Arduino envoie un message sur le moniteur série on a cette fonction :

```
Serial.print("Hello les makers ");
```

## Faire communiquer le PC et la carte Arduino: Liaison série

Voilà ce que donne le code complet :

```
void setup() {
      // mettre ici le code qui sera répété qu'une fois:
Serial.begin(9600); // Permet d'initialiser le moniteur série à 9600 bauds
Serial.print("Hello les makers "); //Permet d'afficher Hello les makers sur le moniteur série
Serial.println(" j'espère que ce tutoriel vous plaît: ");
Serial.print("          signé : " );
Serial.println("IoT-LSI3" );
}
void loop() {
// On ne fait strictement rien !
}
```

```
void loop() {
// mettre ici le code qui sera répété en boucle:
if (Serial.available() )
{
    //Si un message a été reçu  faire ceci
    Serial.println(Serial.readString()); // Lit le message reçu
}                                        //et l'affiche sur le moniteur
else
{
 // Si rien n'est reçu faire cela
}
}
```

# 3.2 Objet IOT 1: Capteur de température

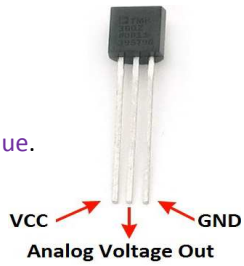
## Acquisition de température : LM35

**Présentation du capteur :**

- Alimenter les pattes *VCC* et *GND*.
- Brancher la patte *centrale* à une entrée analogique.
- Températures comprises entre -55°C à +150°C

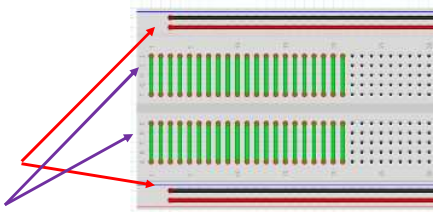
**Lecture du capteur :**

- *10mV ⇨ 1°C*
- La lecture analogique d'un signal de 0 à 5V étant codée de 0 à 1023.
- ***Temp** = **Volt***(5/1023)*100*
- ***Volt*** est entre 0 et 1023




```
float temp;
void setup()
{ Serial.begin(9600);
}
void loop()
{ temp =
analogRead(A0)*(5.0/1023.0*100.0);
  Serial.println(temp);
  delay(1000);
}
```

**Fonctionnement de la *plaque d'essaie* (*breadboard*)**



- Permet de relier facilement les composants entre eux.
- La plaque est composée de deux zones :
  - Les bandes de chaque côté : On branche le VCC et le GND.
  - La zone centrale : On branche les composants entre eux.
  - Sur le schéma, chaque trait de couleur représente un ensemble de trous reliés entre eux.

# 3.3 Objet IOT 2: Contrôle d'accès avec RFID

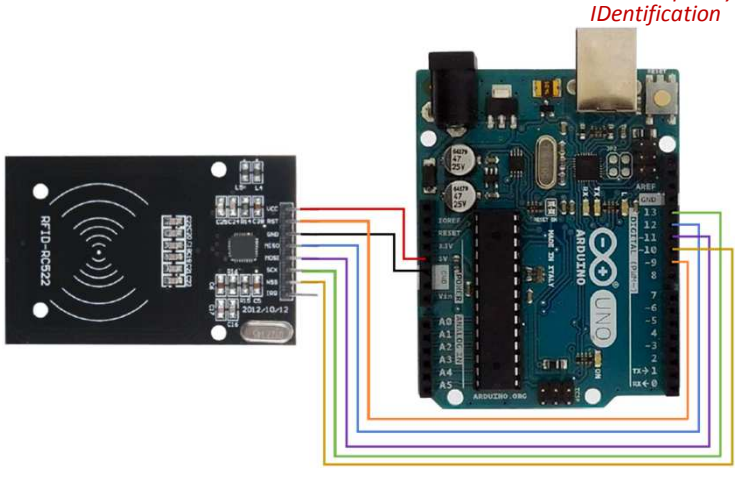



*Radio Frequency IDentification*

## Connecter le module *MFRC522*

- Branchez le module LCD aux broches de l'Arduino comme représenté ci-dessous :
- Table de correspondance

| ***RFID-RC522*** | ***Arduino*** |
|---|---|
| VCC | 3.3V |
| RST | 9 |
| GND | GND |
| MISO | 12 |
| MOSI | 11 |
| SCK | 13 |
| NSS | 10 |
| IRC | / |

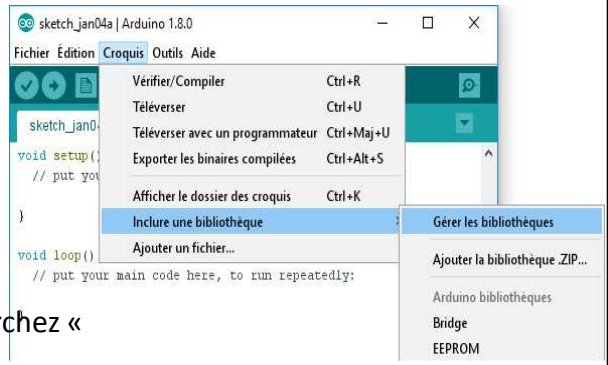


## 3.3 Objet IOT 2: Contrôle d'accès avec RFID

### Installation de la bibliothèque

Pour pouvoir utiliser ce module, la bibliothèque *MFRC522* doit être installée:

- Ouvrez l'IDE Arduino et aller dans *Croquis* ⇨ *Inclure une bibliothèque* ⇨ *Gérer les bibliothèques*
- Dans le gestionnaire de bibliothèques, recherchez « *MFRC522* » et installez *MFRC522*:



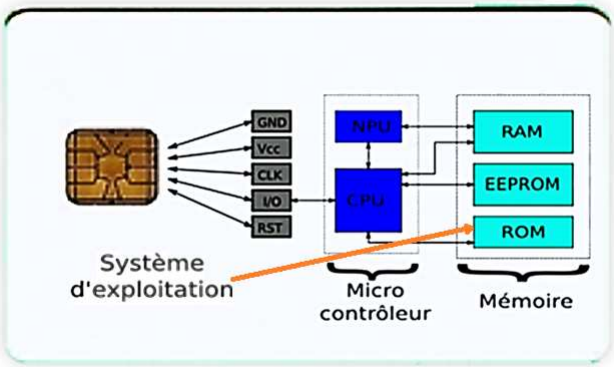




*Fermer la fenêtre une fois l'installation terminée*

### Exemple de programme

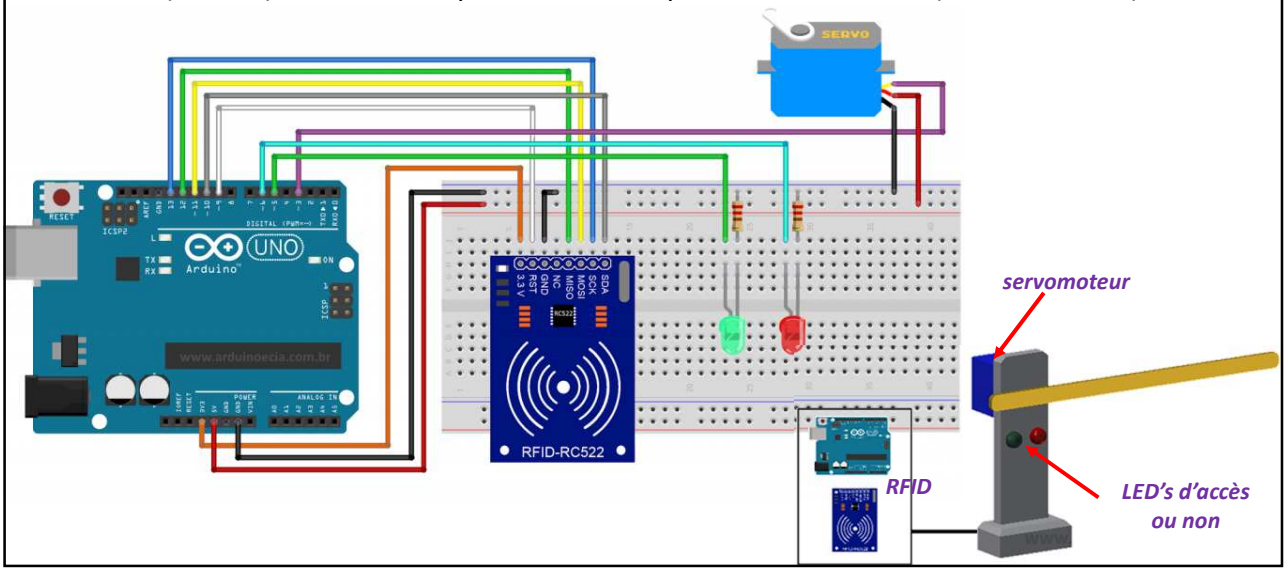
L'exemple de code suivant (à copier dans l'IDE Arduino) initialise le module RFID et affiche les données de la carte ou du badge présenté sur le moniteur série (*CTRL+MAJ+M* pour ouvrir le moniteur série) :




```
#include <SPI.h>
#include<MFRC522.h>
// Affectation des broches
#define RST_PIN 9
#define SS_PIN 10
MFRC522 mfrc522(SS_PIN, RST_PIN);
void setup() {
  // Initialisation du Module RFID
  Serial.begin(9600);
  while(!Serial);
  SPI.begin();
  mfrc522.PCD_Init();
  mfrc522.PCD_DumpVersionToSerial(); // Affichage
  des données de la bibliothèque
  Serial.println(F("Scan PICC to see UID, type, and
  data blocks..."));  }
void loop() {
  // Attente d'une carte RFID
  if( ! mfrc522.PICC_IsNewCardPresent()) {
  return; }
  // Récupération des informations de la carte RFID
  if( ! mfrc522.PICC_ReadCardSerial()) {
  return; }
  // Affichage des informations de la carte RFID
  mfrc522.PICC_DumpToSerial(&(mfrc522.uid)); }
```

## 3.3 Objet IOT 2: Contrôle d'accès avec RFID

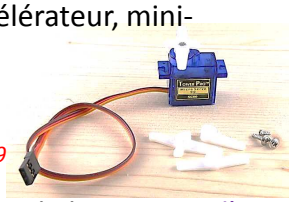
Nous allons tester le module RFID en assemblant un contrôle d'accès qui vérifie le droit d'accès (ou non) à travers une porte contrôlée par un servomoteur (microservo 9G):




## 3.3 Objet IOT 2: Contrôle d'accès avec RFID

### Contrôle d'un servomoteur

- Un moteur qui peut tourner avec une liberté de 180° et garder de manière précise l'angle de rotation.
- Utilisé pour contrôler des systèmes mécaniques (gouverne d'avion, accélérateur, mini-robots, etc.).
- Il existe divers types de taille, poids et couple (force) différents.



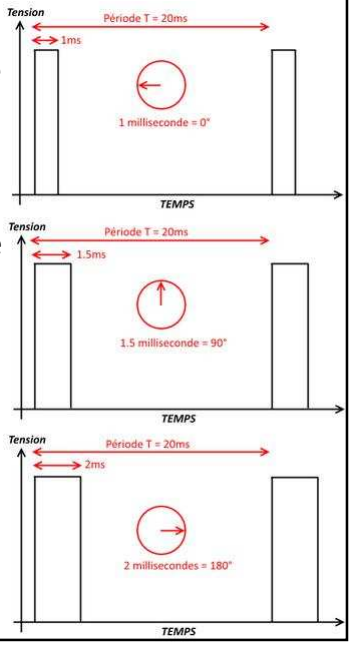
*Servomoteur chinois "9 grammes"*

### Fonctionnement

- Un circuit électronique contrôle un moteur à courant continu en fonction de la *position d'un potentiomètre* intégré au servomoteur.
- Quand le moteur tourne, il entraine avec lui le potentiomètre. Le circuit électronique *ajuste* continuellement *la vitesse* du moteur pour que le potentiomètre (et par extension le bras) reste au même endroit.
- Il suffit de donner une consigne au servomoteur ("reste à 45°" par exemple) et le servomoteur fera son maximum pour rester au plus près de cette consigne.

# Fonctionnement et contrôle d'un servomoteur

- Cette consigne est transmise au moyen d'une impulsion numérique (*PWM*).
- Pour que le servomoteur reste à une position donnée, il faut transmettre toutes les 20 millisecondes (50Hz) une impulsion d'une longueur comprise entre 1 et 2 millisecondes.
  - Une impulsion de 1 milliseconde correspond à 0°.
  - Une impulsion de 2 millisecondes correspond à 180.
  - En envoyant une impulsion d'une longueur intermédiaire, on obtient des angles différents.




# Fonctionnement et contrôle d'un servomoteur

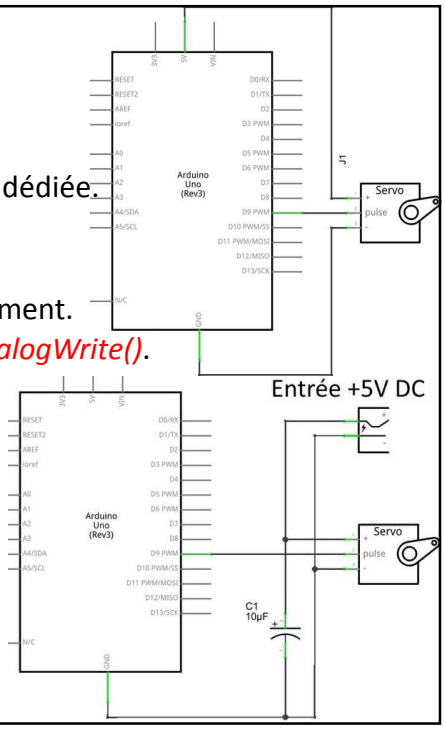
## *Montage*

- **Attention:** à la consommation électrique du servomoteur
- **solution**: câbler les servomoteurs à une alimentation 5 volts dédiée.

## *Code*

- La bibliothèque de code à utiliser est nommée *Servo*.
- Elle permet de contrôler jusqu'à 12 servomoteurs simultanément.
- Elle rend inutilisable les broches D9 et D10 en PWM avec *analogWrite()*.

```
#include <Servo.h>
 Servo monServomoteur; //créer un objet Servo
void setup() { monServomoteur.attach(9); } //numéro de broche sur
void loop() {                          // laquelle le servomoteur est câblé
      // Fait bouger le bras de 0° à 180°
       for (int position = 0; position <= 180; position++) {
          monServomoteur.write(position);
          delay(15);          }
      // Fait bouger le bras de 180° à 10°
      for (int position = 180; position >= 0; position--) {
         monServomoteur.write(position);
         delay(15);        } }
```



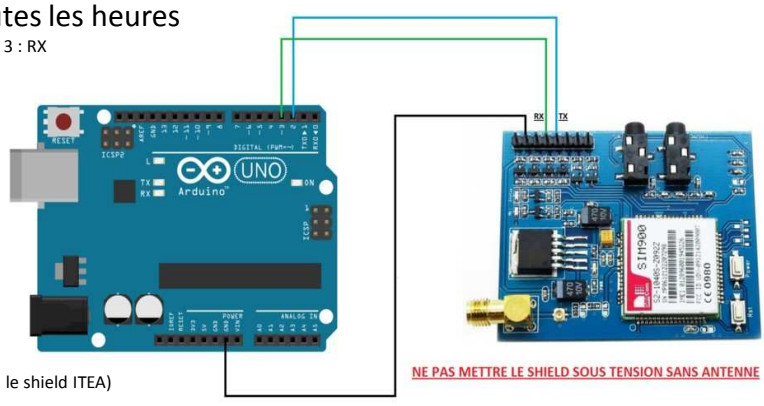
## Contrôle d'accès avec RFID

```
#include <SPI.h>
#include <MFRC522.h>
#include <Servo.h>
Servo microservo9g;
#define SS_PIN 10
#define RST_PIN 9
MFRC522 mfrc522(SS_PIN, RST_PIN);
int led_libéré = 5;
int led_refusé = 6;
char st[20];
void setup() {
pinMode(led_libéré, OUTPUT);
pinMode(led_refusé, OUTPUT);
microservo9g.attach(3);
microservo9g.write(90);
Serial.begin(9600);
SPI.begin();
mfrc522.PCD_Init();
Serial.println(" Rapprochez votre carte du lecteur...");
Serial.println(); }
void loop() {
 if ( ! mfrc522.PICC_IsNewCardPresent()) {
 return; }
 if ( ! mfrc522.PICC_ReadCardSerial()) { return; }
 Serial.print(" Numéro d'identification UID:");
 String contenu= "";
 byte lettre;
for (byte i = 0; i < mfrc522.uid.size; i++) {
   Serial.print(mfrc522.uid.uidByte[i] < 0x10 ? " 0" : " ");
   Serial.print(mfrc522.uid.uidByte[i], HEX);
contenu.concat(String(mfrc522.uid.uidByte[i] < 0x10 ? " 0" : " "));
 contenu.concat(String(mfrc522.uid.uidByte[i], HEX)); }
 Serial.println();
 Serial.print(" Publier : ");
 contenu.toUpperCase();
 if (contenu.substring(1) == "D8 4B 12 22") {
   microservo9g.write(-90);
   digitalWrite(led_libéré, HIGH);
   Serial.println(" Carte 1 - Accès débloqué !");
   Serial.println(); delay(3000);
   microservo9g.write(90);
   digitalWrite(led_libéré, LOW); }
   if (contenu.substring(1) == "87 4B DC 8A") {
     Serial.println(" Carte 2 - Accès refusé !!");
     Serial.println();
     for (int i= 1; i<5 ; i++) {
       digitalWrite(led_refusé, HIGH);
       delay(200);
       digitalWrite(led_refusé, LOW);
       delay(200); }
     }
     delay(1000);
 }
```

## 3.4 Objets IOT connectés: Via un Shield GSM/GPRS

```
//Programme d'envoi d'un sms toutes les heures
//Shield GSM/GRPRS ITEA //Arduino uno //Pin 2 : TX //Pin 3 : RX
//Carte sim sans code pin
#include "SIM900.h"
#include <SoftwareSerial.h>
#include "sms.h"
SMSGSM sms;
int numdata;
boolean started=false;
char smsbuffer[160];
char n[20];
char num_tph[] = "+21698600000";
void setup(){
   //Serial connection.
   Serial.begin(9600);
   Serial.println("GSM Shield testing.");
   //Start configuration of shield with baudrate 2400(pour le shield ITEA)
   if (gsm.begin(2400)) {
      Serial.println("\nstatus=READY");
      started=true;
   } else Serial.println("\nstatus=IDLE");

   if(started) {
      if (sms.SendSMS(num_tph, "start"))
      Serial.println("\nSMS sent OK");    }};
void loop(){
   if(started) {
    sms.SendSMS(num_tph, "message");
      delay(3600000);    }};
```



## 3.4 Objets IOT connectés: Via un WiFi ESP8266

**Connecter Arduino à l'Internet en utilisant le module WiFi ESP8266**

- Le module Wifi ESP8266 (*ESP-01*) est un réseau WiFi complet où on peut facilement nous connecter, en tant qu'adaptateur Wi-Fi de service, grâce à sa connectivité via la communication série ou l'interface UART.
- Il peut être programmé en utilisant directement les ***commandes AT*** (Abréviation de ATtention), le ***code Arduino*** ou encore le ***micrologiciel python***,
- **Attention**: Sur le web, des nombreuses informations sont périmées ou tout simplement fausses.

| *Commandes* | *Description* | *Type* |
|---|---|---|
| AT+RST | Restart module | basic |
| AT+CWMODE | Wifi mode | wifi |
| AT+CWJAP | Join AP | wifi |
| AT+CWLAP | List AP | wifi |
| AT+CWQAP | Quit AP | wifi |
| AT+CIPSTATUS | Get status | TCP/IP |
| AT+CIPSTART | Set up TCP or UDP | TCP/IP |
| AT+CIPSEND | Send data | TCP/IP |
| AT+CIPCLOSE | Close TCP or UDP | TCP/IP |
| AT+CIFSR | Get IP | TCP/IP |
| AT+CIPMUX | Set multiple connections | TCP/IP |
| AT+CIPSERVER | Set as server | TCP/IP |

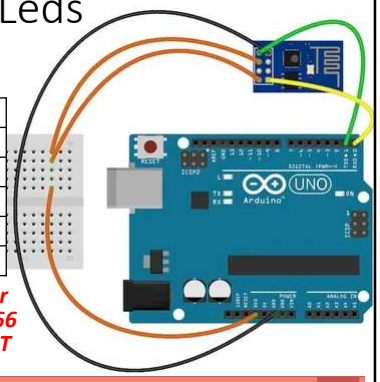
## Application IOT1: Contrôle à distances des Leds

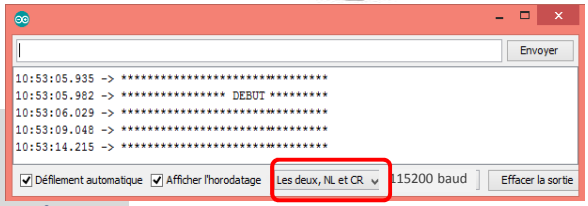
**Méthode 1**: parler au module manuellement via Arduino

- L'ESP8266 n'est **strictement** alimenté qu'en 3,3 V.
- Téléverser le Sketch *Blink* pour allouer la liaison série,
- Régler le terminal pour envoyer CR et LF en caractère de fin de ligne et le débit à 115200 baud. Puis envoyer la commande AT (suivie d'un retour à la ligne).

| *ESP8266* | *Arduino* |
|---|---|
| GND | GND |
| VIN | 3.3v |
| ENABLE | 3.3v |
| TX | RX |
| RX | TX |

***Schéma pour initialiser manuellement l'ESP8266 avec les commandes AT***






```
AT+RST // remise à zéro
AT+CWMODE=3 // 3:mode Client/Serveur, 2:mode AP
AT+CWLAP // lister les point d'accès AP
AT+CWJAP="nom du réseau", "mot de passe" //connecter au wifi
AT+CIFSR //connaitre l'adresse IP de l'ESP
```

# Application IOT1: Contrôle à distances des Leds

**Méthode 2**: parler au module automatiquement

- Il nous faut au minimum deux serials hardware (un pour le pc, et un pour l'ESP) ⇨ Solution: utiliser les GPIO 10 et 11.

| ESP8266 | Arduino |
|---|---|
| GND | GND |
| VIN | 3.3v |
| ENABLE | 3.3v |
| TX | GPIO10 |
| RX | GPIO11 |

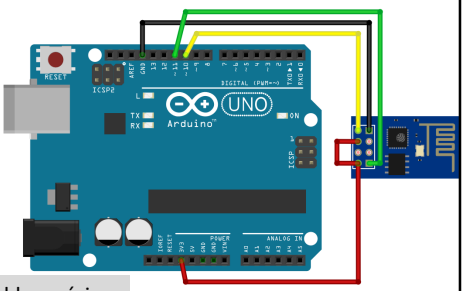


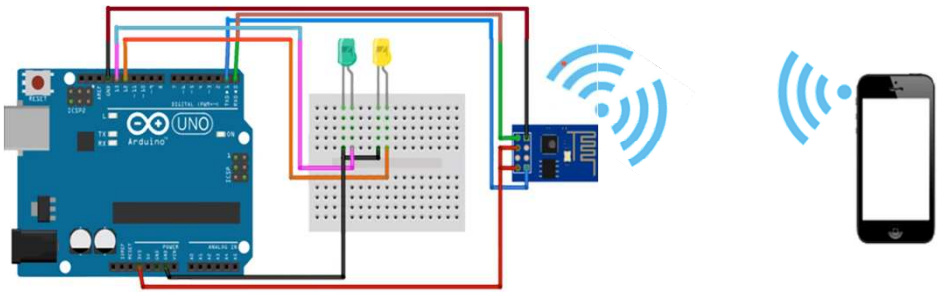
*Schéma pour initialiser automatiquement l'ESP8266*

```
SoftwareSerial ESP8266(10, 11); // configurer les Pin 10 et 11 comme un second bus série
Serial.begin(115200);
ESP8266.begin(115200);  // activation de la 2ième communication série
WiFi.begin("network-name", "pass-to-network"); //connecter le module ESP au Wi-Fi
Serial.println(WiFi.localIP());
svr=net.createServer(net.TCP) // création d'un serveur web
srv:listen(80,function(conn)
Conn:on("receive",function(client,request)
….
ESP8266.println("AT+RST");      // envoie d'une commande AT à l'ESP
Serial.print(ESP8266.read());  // lecture (par caractères) de la réponse du ESP
```

***Nous sommes maintenant prêts à démarrer le WIFI avec l'Arduino UNO.***

# Application IOT1: Contrôle à distances des Leds

- L'ESP démarre en mode *Point d'Accès*, on peut donc s'y connecter à l'aide d'un smartphone pour un contrôle à distance.
- Séquence vidéo (*https://www.youtube.com/watch?v=2cjufbgOBYo*) illustre les étapes à réaliser de l'application.
- Environnement de développement utilisé est ***Remotexy***.
- Code voir: *https://goo.gl/cxptgA#stay_creative #nothing_impossible*



# 3.5 Diagrammes fonctionnels

## Rappels

- Les appareils qui se connectent aux réseaux ***WiFi*** sont appelés ***stations*** (STA).
- La connexion au ***WiFi*** est assurée par un ***point d'accès*** (***AP***), qui fait office de ***concentrateur*** pour une ou plusieurs stations.
- De l'autre extrémité, l'***AP*** est connecté à un ***réseau câblé***.
- Pour fournir un accès à l'Internet, l'***AP*** intègre généralement un ***routeur***.
- Chaque ***AP*** est reconnu par un ***SSID*** (***S**ervice **S**et **ID**entifier*), qui est le ***nom du réseau*** qu'on sélectionne lors de la connexion.
- En cas de perte de connexion ou redémarrage, l'ESP8266 se reconnectera automatiquement au dernier ***AP*** utilisé (les informations sont enregistrées dans sa mémoire flash).

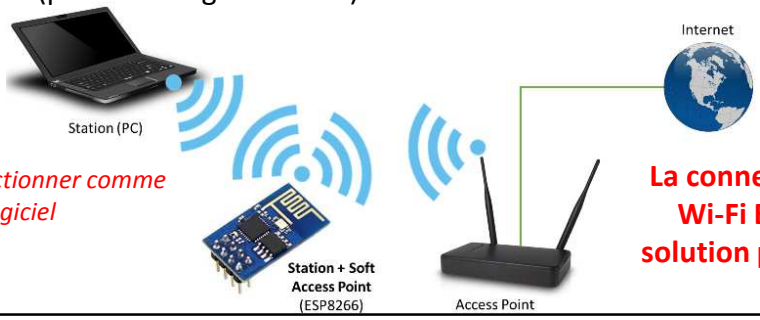
*L'ESP8266 fonctionner en mode **station***




# Diagrammes fonctionnels: Point d'accès logiciel

- L'ESP8266 peut fonctionner comme une ***station***, pour se connecter à un réseau WiFi.
- Il peut aussi fonctionner comme un ***AP*** logiciel (***soft-AP***), pour établir son propre réseau ***WiFi*** et connecter d'autres ***ST*** au module ESP
- L'ESP8266 peut également fonctionner à la fois comme ***station*** et ***soft-AP***. Cela offre la possibilité de construire un ***réseaux maillés***.
- Le nombre maximum de ***ST*** pouvant être connectées simultanément au ***soft-AP***, est par défaut, égal à 4 (peut être réglé de 0 à 8).

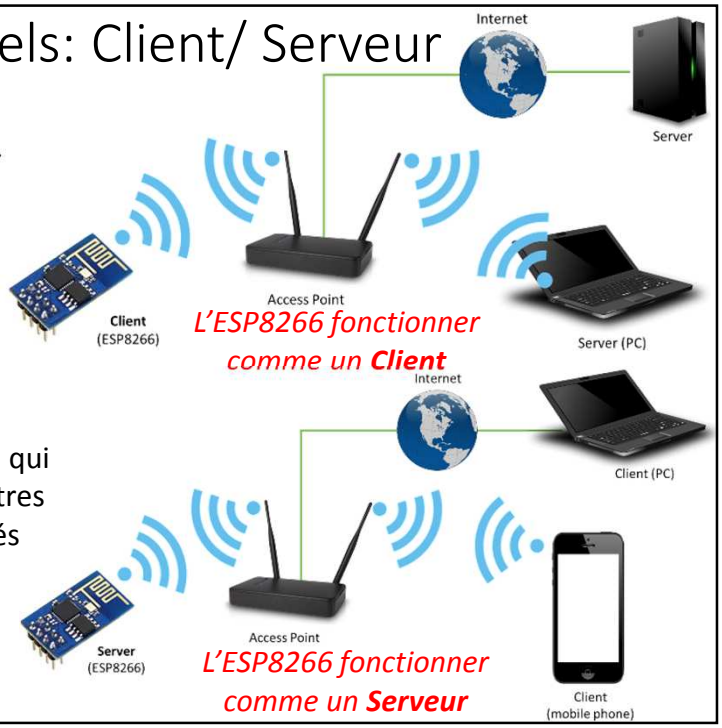
*L'ESP8266 fonctionner comme **station** + **AP** logiciel*




**La connexion entre modules Wi-Fi ESP8266 offre une solution pour les applications IOT**

# Diagrammes fonctionnels: Client/ Serveur

La classe ***Client*** crée des clients qui peuvent accéder aux services fournis par les serveurs afin d'envoyer, recevoir et traiter des données.

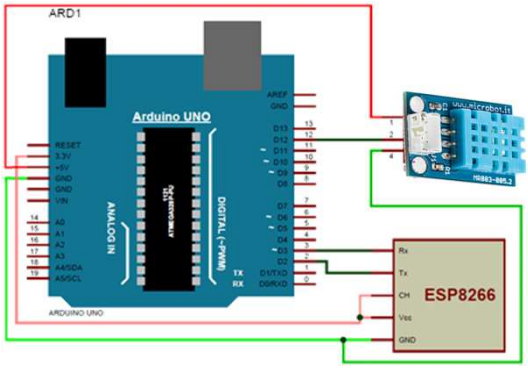



- La classe de ***serveur*** crée des serveurs qui fournissent des fonctionnalités à d'autres programmes ou périphériques, appelés ***clients***.

## 3.6 Application IOT-2: Station météo à distance

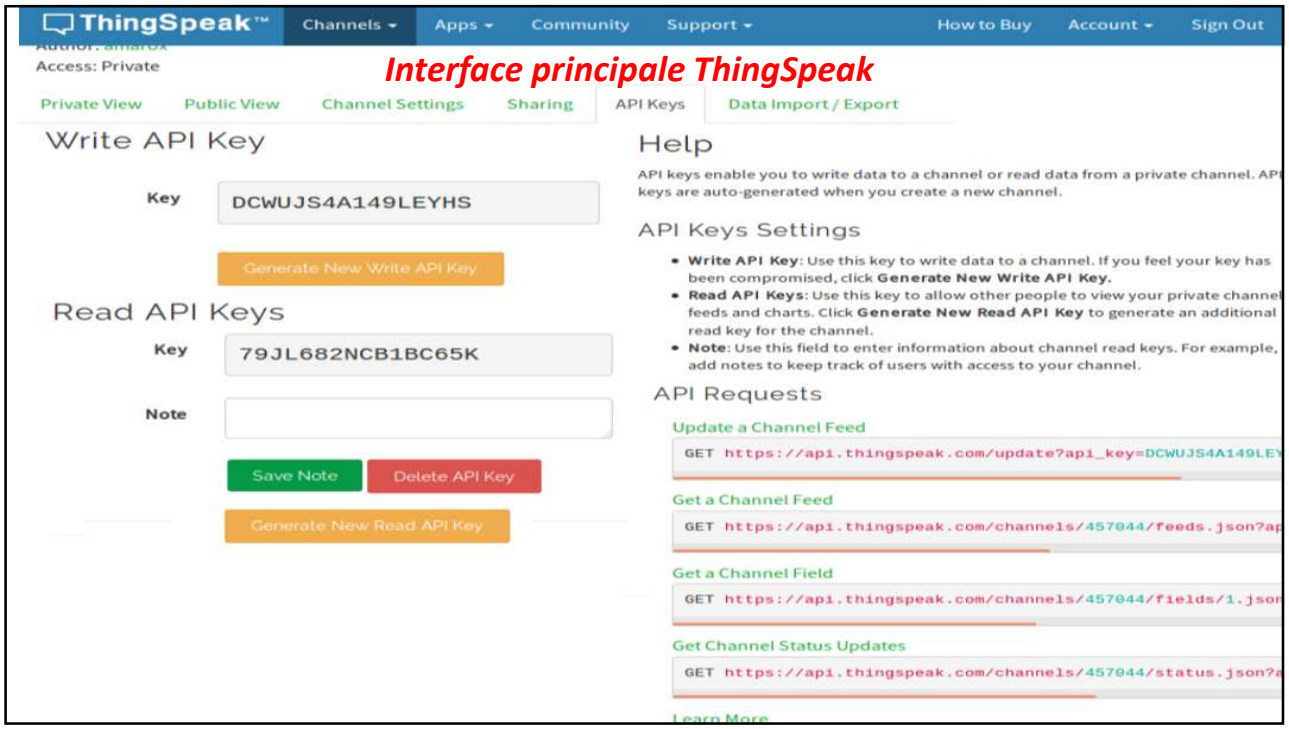
- Station météo à distance (humidité et température) en utilisant **ThingSpeak**,
- ***ThingSpeak*** est un service Cloud gratuit de surveillance et de contrôle des systèmes à distance.
- Les composants à utiliser: Carte Arduino, capteur d'humidité et de température **DHT11** et le ESP8266 WIFI ,

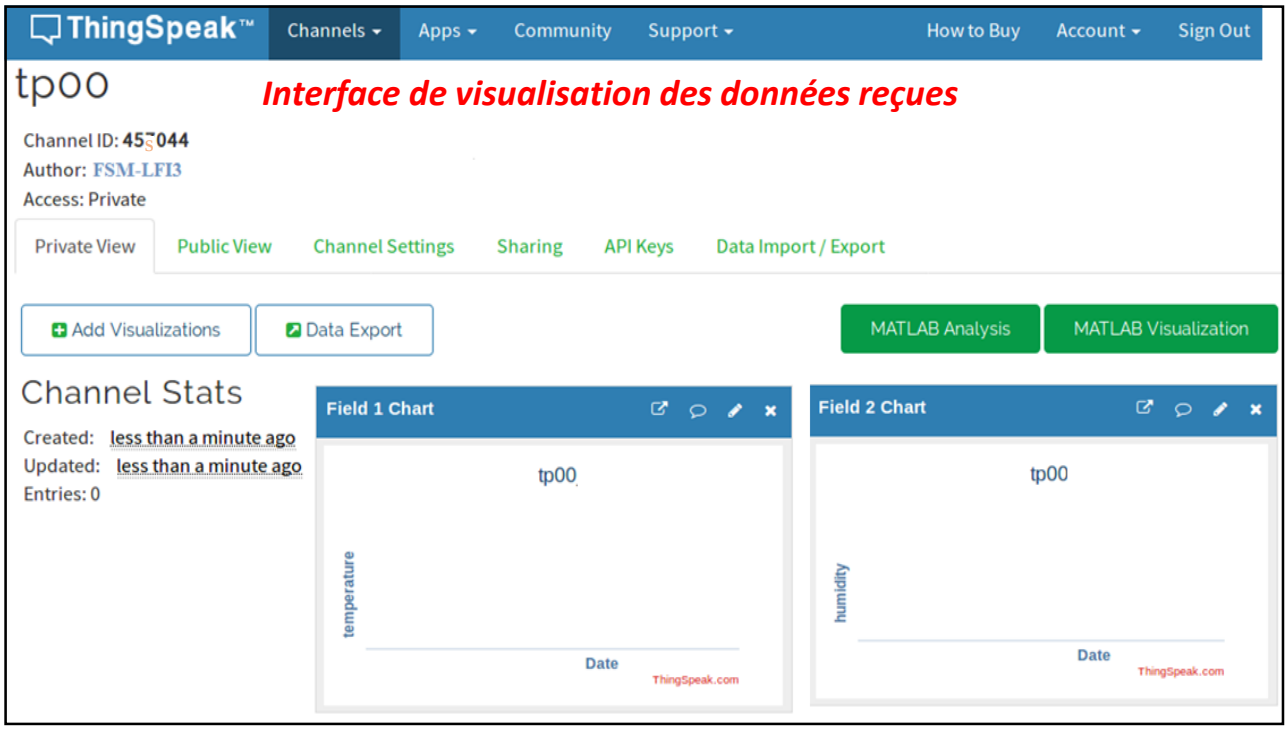
- Le module **DHT11** est connecté au GPIO **12**,
- L'Arduino recueille les données en deux parties: l'humidité puis la température,
- Les broches Tx et Rx de l'ESP8266 sont directement connectées aux GPIO **2** et **3**,
- La bibliothèque "Software Serial Library" est utilisée pour permettre la communication série sur les GPIO **2** et **3** d'Arduino.



## 3.6 Application IOT-2 : Station météo à distance

1. Coté *ThingSpeak.com*, on doit créer un compte pour se connecter et cliquer sur l'option "**New Channel**" dans le menu "**Channels**".
2. Dans le formulaire et avant d'enregistrer la chaîne, cocher les cases **champs** 1 et 2 (**Field** 1 et 2), remplir «**Humidity**» et «**Temprature**» dans les deux étiquettes et cocher la case «**Rendre public**» (Make public). Maintenant, notre nouvelle chaîne est créée.
3. Cliquer sur l'onglet «**API Key**» et récupérer la clé **API d'écriture** pour l'utiliser après, dans le champ **char * api_key** du code.
4. Ensuite, cliquer sur « **Data Import/Export** » et copier l'URL dans « Update Channel Feed GET Request », qui est: https://api.thingspeak.com/update?api_key=XXXXX&field1=0.
5. Coté code, on ouvre «api.thingspeak.com» à l'aide de la fonction http Get, avec postUrl pour envoyer les données via la requête GET au serveur.
6. Pour tester le fonctionnement des liens, utiliser un navigateur et lancer par exemple le lien suivant : ***https://api.thingspeak.com/update?api_key=XXXXX&field1=31***. Dans l'onglet ***Private View*** la valeur 31 s'affiche dans le graphe quelques secondes après avoir lancé le lien.
7. Pour recevoir les données, sous forme de graphique, sur le smartphone via ThingSpeak, on utilise l'application Virtuino, disponible sur Google Play.






# 3.7 Application IOT-3: Smart farming

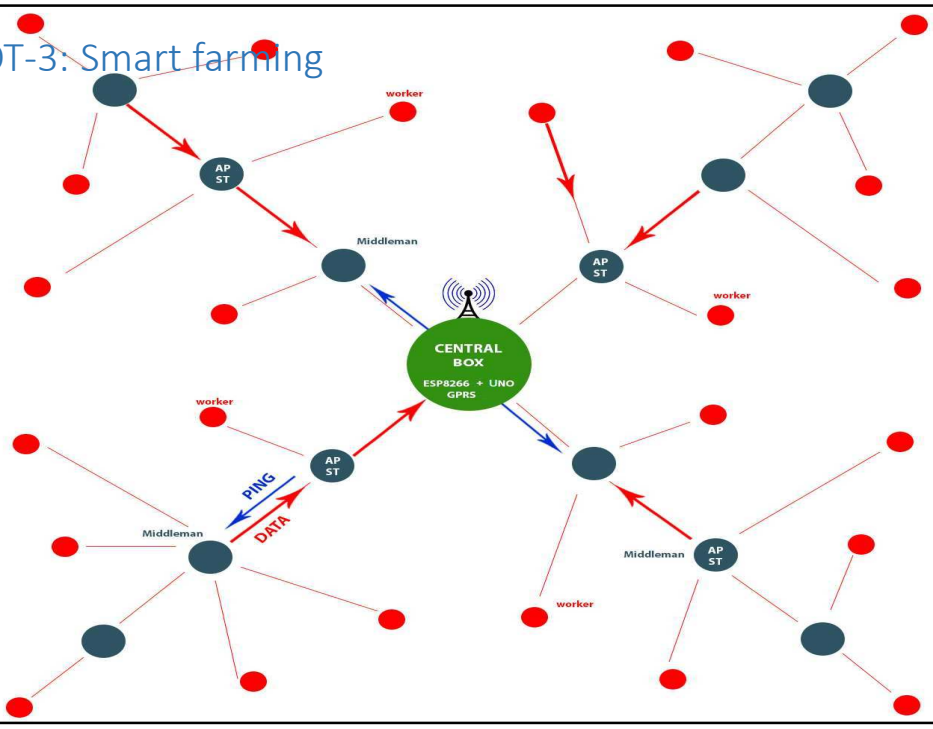
**Problème**

Un agriculteur, qui cultive des légumes sur un champ de 5 kilomètres carrés, cherche à mettre en œuvre un système Wi-Fi de surveillance de la *température* et de *l'humidité* entièrement indépendant, autonome et sans utiliser des fils.

**Solution**: Création d'un réseau Wi-Fi ESP8266

- Diviser le champ en 4 parties de zones,
- Dans chaque zone, on construit un *sous-réseau* de ***9 ESP8266***,
- Au **centre** du terrain, on place un **boîtier central** (*central Box*) contenant
  - 1 module ESP8266,
  - 1 Arduino uno,
  - 1 module compatible GPRS Arduino.
- Dans chaque *sous-réseau*, on dispose :
  - 3 ESP comme **répéteurs** de signaux (*middleman*)
  - 6 autres ESP pour les **lectures** des capteurs (*workers*).

## 3.7 Application IOT-3: Smart farming




*Création d'un réseau WiFi ESP8266 en topologie hybride (étoile –maillé)*

## 3.7 Application IOT-3: Smart farming

- Les 6 *workers* sont, la plupart du temps, en *DEEP_SLEEP* et se réveillent à **1 heure** d'intervalle pour **lire** les capteurs,
- Les 3 *middleman* sont à la fois en ***AP*** et ***ST*** avec un *serveur Web* démarré, en attente des requêtes à venir des *workers*.

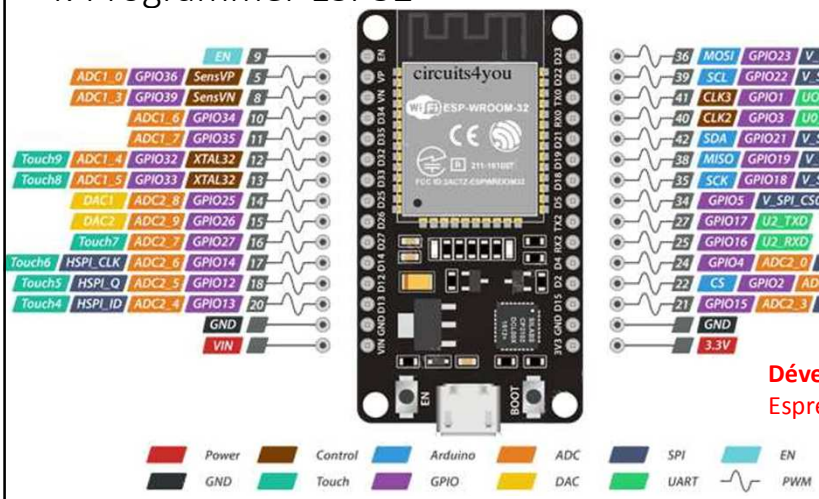
**Comment programmer les modules?**

- Chaque *worker* est programmé pour se réveiller de *DEEP_SLEEP*, **lire** les capteurs, se connecter à l'*AP* ESP le *plus proche* et envoyer une *requête HTTP* à son *serveur* avec les données,
- Si l'*AP middleman* le plus proche n'était pas disponible, les *workers* sont programmés pour conserver jusqu'à **10 lectures** dans leurs **mémoires internes**, et les envoyer chaque fois que l'*AP* est **réveillé**,
- Les *workers* peuvent être alimentés par ***4*** piles *Li-Ion 3,3 V*, branchés en **parallèle** et chargés par des panneaux solaires de 15 V,
- Les *capteurs* à utiliser sont à **1 fil**.

## 3.7 Application IOT-3: Smart farming

- Chaque *middleman* est programmé pour *écouter* et *recevoir* les lectures des *workers*, puis les envoyer au prochain *middleman* jusqu'à l'ESP du *central Box*,
- Les *middlemans* sont configurés à la fois en *AP*/*ST*.
- Les *workers* (*client*) utilisent des adresses *IP statiques* afin de réduire le temps nécessaire pour se connecter à l'*AP*.
- Le *central Box* est chargé de recevoir et de traiter les données des *middlemans* les plus proches, et de les envoyer dans une base de données *MySQL* via le module *GPRS* au format *JSON / XML*, **une fois** par **jour**.
- Les données reçues (toutes les heures) peuvent être stockées (en format *CSV*) sur un **module** de *carte SD*, attaché au **module** *Arduino UNO*.
- Les ESP *central* et *middlemans* sont programmés comme un *agent de découverte*, en envoyant des requêtes *PING* vers les *AP* à proximité pour vérifier la disponibilité et **envoyer** *l'état* et les *mesures* à la base de données.

## 4. Programmer ESP32




- ESP32 est une évolution de l'ESP8266, intégrant la gestion du Wi-Fi et du Bluetooth, apprécié dans le domaine de l'internet des objets.

**Développement**: **Esplorer** (ESP-IDF: Espressif IoT Development Framework),

- Connecter l'ESP32-WROOM à l'ordinateur avec un câble **micro-usb** et Valider le port **COM** utilisé sur l'IDE Arduino en sélectionnant ***Outils/Port Com***
- Ajouter dans <*URL de gestionnaire de carte supplémentaires*> de **Fichier/Préférences** : https://dl.espressif.com/dl/package_esp32_index.json

## 4.1 Programmer ESP32 en utilisant IDE Arduino

- Vérifier et/ou modifier les ***paramètres de la carte*** Esp32 comme indiqué ci-dessous
  - *Type de carte: ESP32 Dev Module*
  - *Upload Speed: 115200 # Vitesse de transfert*
  - *CPU Frequency: 240MHz (WIFI/BT)*
  - *Flash Frequency: 80MHz*
  - *Flash Mode: QIO*
  - *Flash Sizes 4MB(32Mb)*
  - *Partition scheme: Default 4MB with spiffs (1.2MB APP/1.5MB SPIFFS)*
  - *Core Debug Level: Rien*
  - *PSRAM: Disabled*
  - *Port: COM4 # Votre n° de port COM - Windows*

## 4.1 Programmer ESP32 en utilisant IDE Arduino

- Pour vérifier le bon fonctionnement du ESP32-WROOM, nous pouvons exécuter le code pour faire clignoter la led de la carte ESP32 (*Led interne = **pin1***)
- Téleverser le code dans l'EPS32-WROOM
- Lorsque l'IDE Arduino affiche "*Connecting ....*« appuyer durant 2 à 3 secondes sur le bouton [*BOOT*] de l'esp32 pour démarrer le téléversement,

```
int Led=1;    # Ajouter int Led=1; (pin1)
void setup() {
  pinMode(Led, OUTPUT);  # Modifier la variable
}
void loop() {
  digitalWrite(Led, HIGH);   # Modifier la variable
  delay(1000);
  digitalWrite(Led, LOW);   # Modifier la variable
  delay(1000);
}
```

### *Vérification du Wifi de la carte ESP32-WROOM*

- Téleverser le code « *Fichier/Exemples (Exemples pour ESP32 Dev Modules) /Wifi/WifiScan* », ouvrir le *Moniteur Série* et vérifier la *vitesse du moniteur série* pour un affichage correct,

## 4.1 Programmer ESP32 en utilisant IDE Arduino

### *Vérification du Wifi avec la carte ESP32-WROOM*

```
#include "WiFi.h"
void setup()
{
  Serial.begin(115200);
  // Set WiFi to station mode and disconnect from an AP if it
  // was previously connected
  WiFi.mode(WIFI_STA);
  WiFi.disconnect();
  delay(100);
  Serial.println("Setup done");
}
void loop()
{
  Serial.println("scan start");
  // WiFi.scanNetworks will return the number of networks found
  int n = WiFi.scanNetworks();
  Serial.println("scan done");
  if (n == 0) {
    Serial.println("no networks found");
  } else {
    Serial.print(n);
    Serial.println(" networks found");
    for (int i = 0; i < n; ++i) {
      // Print SSID and RSSI for each network found
      Serial.print(i + 1);
      Serial.print(": ");
      Serial.print(WiFi.SSID(i));
      Serial.print(" (");
      Serial.print(WiFi.RSSI(i));
      Serial.print(")");
      Serial.println((WiFi.encryptionType(i) == WIFI_AUTH_OPEN)?"
":"*");
      delay(10);
    }
  }
  Serial.println("");
  // Wait a bit before scanning again
  delay(5000);
}
```

## 4.1 Programmer ESP32 en utilisant IDE Arduino

### *Se connecter au réseau WiFi*

- Le sketch suivant est utilisé pour installer et vérifier la connexion à notre réseau WiFi. Il faudra indiquer le *nom de son routeur* WiFi (la variable *ssid*) et le *mot de passe* (la variable *pwd*).

```
#include "WiFi.h"
const char* ssid = "...";
const char* pwd  = "...";
void setup() {
  Serial.begin(9600);
  Serial.println("Configuration du WiFi");
  WiFi.begin(ssid, pwd);
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.println("Connexion au WiFi..");
  }
  Serial.println("Connecté au réseau WiFi");
}
void loop() {
  delay(1000);
  if (WiFi.status() == WL_CONNECTED) {
    Serial.print("Est connecté au réseau avec ");
    Serial.println(WiFi.localIP());
  }
  else {
    Serial.println("N'est pas connecté au réseau");
  }
}
```

## 4.1 Programmer ESP32 en utilisant IDE Arduino

### *Un serveur Web simple*

- L'accès se fait avec par exemple http://192.168.1.7/test.
- Voici le script :

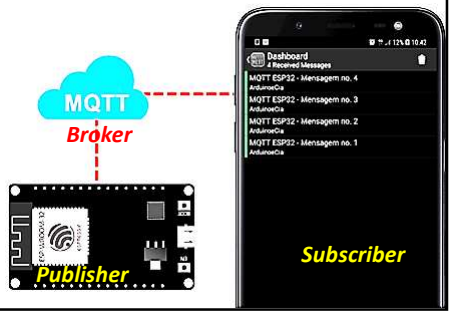
```
#include <WiFi.h>
#include <WebServer.h>
const char* ssid = "...";
const char* pwd  = "...";
WebServer server(80);
void setup() {
 Serial.begin(9600);
 Serial.print("Connexion à ");
 Serial.println(ssid);
 WiFi.begin(ssid, pwd);
 while (WiFi.status() != WL_CONNECTED) {
  delay(200);
  Serial.print(".");
 }
 //Montre l'adresse IP et démarre le serveur Web
 Serial.println("");
 Serial.println("WiFi connecté!");
 Serial.println("Adresse IP: ");
 Serial.println(WiFi.localIP());
 server.on("/", deal_onConnect);
 server.on("/test", deal_aTest);
 server.on("/temps", deal_temperatures);
 server.onNotFound(deal_notFound);
 server.begin();
}
void deal_onConnect() {
 Serial.println("onConnect");
 server.send(200, "text/html", SendHTML("Connect sur root"));
}
void deal_aTest() {
 Serial.println("aTest");
 server.send(200, "text/html", SendHTML("Juste un test"));
}
void deal_temperatures() {
 Serial.println("The temperatures");
 server.send(200, "text/html", "C: 20.6 17.8 22.5");
}
void deal_notFound() {
 Serial.println("notFound");
 server.send(200, "text/html", SendHTML("Not found"));
}
String SendHTML(String content){
 String ptr = "<!DOCTYPE html> <html>\n";
 ptr += "<head><meta name=\"viewport\" content=\"width=device-width, initial-scale=1.0, user-scalable=no\">\n";
 ptr += "<title>ESP32 Web Server</title>\n";
 ptr += "<style>html { font-family: Helvetica; display: inline-block; margin: 0px auto; text-align: center;}\n";
 ptr += "body{margin-top: 50px;} h1 {color: #444444;margin: 50px auto 30px;} h3 {color: #4444FF;margin-bottom: 50px;}\n";
 ptr += "p {font-size: 14px;color: #888;margin-bottom: 10px;}\n";
 ptr += "</style>\n</head>\n";
 ptr += "<body>\n";
 ptr += "<h1>ESP32 Web Server</h1>\n";
 ptr += "<h3>" + content + "</h3>\n";
 ptr +="</body>\n";
 ptr +="</html>\n";
 return ptr;
}
void loop() {
 server.handleClient();
 delay(1000);
}
```

## 4.2 Application IOT-4: Envoi d'un messages MQTT

- Il s'agit d'envoyer un message **MQTT** en utilisant le module ESP32,
- Le message est envoyé à un smartphone Android en utilisant le **cloud**,
- A la place de l'ESP32, on peut également utiliser la carte Arduino+ ESP8266.

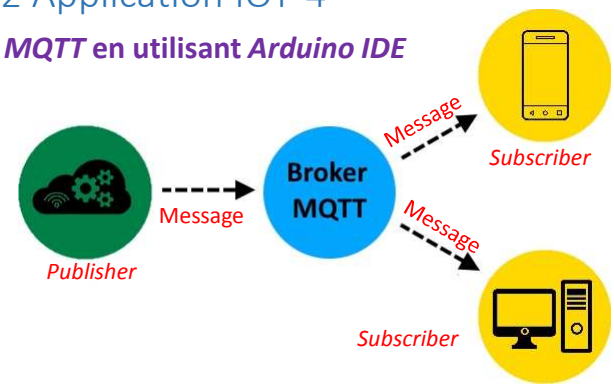
### Comment fonctionne le MQTT

- **MQTT** (Message Queue Telemetry Transport) est un **protocole de communication** simple à utiliser et ne nécessite pas beaucoup de puissance de traitement ou de grandes ressources de bande passante réseau.
- Fondamentalement, il y'a un
  - ***Editeur*** (*Publisher*), qui *envoie des messages*;
  - ***Serveur MQTT*** (*Broker*), à mi-chemin, responsable de la *gestion des échanges de messages*,
  - ***Abonné*** (*Subscriber*), qui *lit des messages* envoyés par *Publisher*.



## 4.2 Application IOT-4

***MQTT* en utilisant *Arduino IDE***




Le *Broker MQTT* utilisé est **iot.eclipse.org**, qui est gratuit. Il existe également des *Brokers* payants sur Internet et on peut même créer un *Broker* en **interne**, en utilisant une carte *Raspberry Pi*, par exemple.

- Installer la carte ESP32 dans l'IDE Arduino. Dans le menu *Fichier->Préférences*, mettre l'adresse ci-dessous dans le champ *URL* pour les gestionnaires de cartes: *https://dl.espressif.com/dl/package_esp32_index.json*
- Accéder au menu *Outils -> Carte -> Gestionnaire de carte* et installer le module *ESP32,*

## 4.2 Application IOT-4

- La bibliothèque de client MQTT fournit des API pour utiliser le protocole MQTT.
- Installer la bibliothèque *PubSubClient* (communication MQTT), en sélectionnant *Croquis -> Ajouter une bibliothèque -> Gérer les bibliothèques* puis *Installer.*
- Sélectionner la carte du module de développement ESP32 dans le menu *Outils*,
- Charger le programme ci-suivant, pour se connecter au réseau wifi et au *serveur MQTT* (*iot.eclipse.org*) et envoyer des messages à des intervalles de 30 secondes,
- Modifier les lignes 4 et 5 du programme avec le *nom* et le *mot de passe* du réseau wifi, à travers lequel l'ESP32 communique avec le *Broker MQTT* pour envoyer des données.

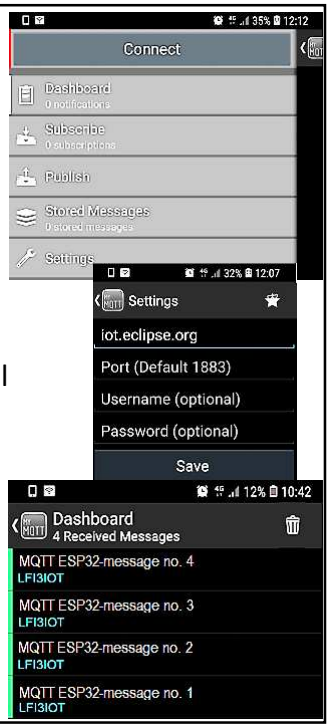
```
//Programme: module ESP32 Wifi avec MQTT
#include <WiFi.h>
#include <PubSubClient.h>
const char* ssid = "WIFI-FSM";
const char* password =  « 12457894";
const char* mqttServer = "mqtt.eclipse.org";
const int  mqttPort = 1883;
const char* mqttUser = "abcdefg";
const char* mqttPassword = "123456";
WiFiClient espClient;
PubSubClient client(espClient);
int compteur = 1;
char message[30];
void setup()
{ Serial.begin(115200);
  WiFi.begin(ssid, password);
  while (WiFi.status() != WL_CONNECTED)
  { delay(500);
    Serial.println("Démarrage de la connexion WiFi..");}
   Serial.println("Connecté au réseau WiFi!");}
void loop()
{  //Se connecte au Broker MQTT
  reconnectbroker();
  sprintf(message, "MQTT ESP32-message no. %d", compteur);
  Serial.print(« Message envoyé: ");
  Serial.println(message);
```

```
//Envoyez le message au broker
 client.publish(« LSI3IOT", message);
Serial.println("Message envoyé avec succès...");
 compteur++;
 delay(30000);}
void reconnectbroker()
{  // Connexion au broker MQTT
 client.setServer(mqttServer, mqttPort);
 while (!client.connected())
 { Serial.println(" Connexion au broker MQTT...");
   if (client.connect("ESP32Client", mqttUser, mqttPassword ))
   {  Serial.println(" Connexion au broker!");  }
   else
   { Serial.print("Échec de la connexion au Broker- État:");
     Serial.print(client.state());
     delay(2000);}
 } }
```

## 4.2 Application IOT-4

### Réception de données MQTT sur le smartphone

- Pour recevoir sur le smartphone, les données envoyées par le module ESP32 via MQTT, on utilise l'application *MyMQTT*, disponible sur *Google Play*,
- Sélectionner l'icône de l'application pour accéder au *menu* puis *Settings* et entrer *iot.eclipse.org*, puis *Save*.
- De retour au *menu*, sélectionner *Subscribe* et entrer le *topic* auquel on souhaite s'abonner. Dans notre cas, le nom du *topic* est **LSI3IOT**. Cliquez sur *Add* pour enregistrer les informations,
- Revenir à l'écran principal (Dashboard) et suivre la réception des messages MQTT envoyés par ESP32,
- A noter, qu'il est possible d'envoyer des commandes à partir du smartphone à l'ESP32.



# 4.3 Principaux Brokers MQTT open-source

Les principaux projets de brokers Open Source

- ***ActiveMQ*** qui permet d'ajouter MQTT à un serveur Web Apache,
- ***JoramMQ*** pour l'intégration de MQTT en Java,
- ***Mosquitto***, le plus utilisé dans les projets DIY soutenu par la fondation eclipse.org,
- ***RabbitMQ***, disponible également avec un support commercial,
- ***EMQTT***, conçu pour recevoir de nombreuses connexions (jusqu'à 1 million par serveur). Il est possible de créer un **cluster** (réseau de serveur) pour accroitre le nombres de connexions simultanées.

***ActiveMQ*** et ***JoramMQ*** sont des brokers assez spécifiques. ***RabbitMQ*** est plus orienté entreprise.

***Mosquitto*** est un très bon choix. Il est disponible sur toutes les plateformes et s'installe en quelques minutes.

# 4.4 Services MQTT Broker gratuits

Les ***brokers*** accessibles au public, utiles pour les tests et le prototypage, sont:

| adresse | port | type | info |
|---|---|---|---|
| iot.eclipse.org | 1883 , 80 (WebSockets) | mosquitto | web page, Xively statistics, topics and HTTP bridge |
| test.mosquitto.org | 1883 , 8883 (SSL), 8884 (SSL), 80 (WebSockets) | mosquitto | web page, Xively statistics, topics and HTTP bridge |
| dev.rabbitmq.com | 1883 | rabbitmq | admin dashboard |
| broker.mqttdashboard.com | 1883 , 8000 (WebSockets) | HiveMQ | information page, stastistics and dashboard |
| q.m2m.io | 1883 | m2m.io | requires signup/username and password, connecting to m2m.io |
| q.m2m.io | 1883 | m2m.io | requires signup/username and password, connecting to m2m.io |
| www.cloudmqtt.com (Rmq: l'hôte réel varie) | 18443 , 28443 (SSL) | mosquitto | nécessite inscription / nom d'utilisateur et mot de passe, prix (plan gratuit disponible), documentation |
| mqtt.simpleml.com | 1883 (MQTT), 8883 (MQTT+SSL), 80 (REST), 80 (WebSockets), 5683 (CoAP) | SimpleML | Service MQTT gratuit pour évaluer les modèles Machine Learning, la documentation |
| mqtt.kgbvax.net | 1883 (MQTT) | mosquitto | Instance partagée gratuite, prend actuellement en charge la persistance |